

RASPUTIN EVOLUTION PROPOSAL

From AI workstation to the Rasputin personal assistant engine

A staged architecture for a self-hosted personality, cross-workspace context, and governed local computer control

Prepared for: Elliott - Rasputin maintainer and operator

Document status: Review draft - no implementation or deployment approved

Scope: Rasputin personality, cross-workspace context, model fleet, speech I/O, local agent control

Design posture: Local process control, privacy-first, evidence-driven, user-controlled

Decision gate: Approve, revise, or reject the direction before iteration begins

REVIEW BOUNDARY

This is a design document only. It proposes an evolution path, contracts, milestones, and decisions for approval. It does not change Rasputin source code, download models, alter Docker instances, commit files, or enable autonomous actions. It also does not grant any model runtime generic API access to the host computer.

1. Executive intent

Rasputin should remain a serious local AI workstation while becoming the product's persistent personality: a private, self-hosted presence that can listen, reason, act, remember, and speak. Rasputin is not synonymous with any one model. The personality, identity, context policy, and user relationship belong to the Rasputin application; specialized local models and agents are replaceable workers behind it.

Rasputin should be able to understand approved context from across chats and workspaces, then turn a natural request into a governed plan. It can delegate to registered agents, stream their status, ask for approvals, and summarize their results. A local control broker may open VS Code, run tests, inspect files, or operate Docker, but model containers receive only scoped requests and structured receipts, never generic access to the personal computer.

The central design choice is to build a coordinated model and agent fleet rather than searching for one oversized model that does everything. A capable local coordinator can route specialized models for planning, coding, transcription, speech synthesis, perception, embeddings, and background work while preserving one stable Rasputin identity and one governed conversation contract.

NORTH-STAR STATEMENT

Rasputin is a local-first personal AI engine: one persistent personality that can recall approved context across chats and workspaces, delegate to specialized local models and agents, and use a local, approval-governed computer bridge without exposing the host computer to a remote API model.

2. Product direction: two modes, one governed core

The workstation and assistant objectives reinforce one another. Workstation mode optimizes for deliberate, reviewable work. Assistant mode optimizes for natural, continuous interaction with the same Rasputin personality. Both use the same policy layer, model fleet, memory boundaries, tool broker, and run receipts.

Dimension	Workstation mode	Assistant mode
Primary interaction	Browser workspace, chat, task panels, files, terminal	Voice or text conversation, push-to-talk, optional wake word
Tempo	Deliberate; plans, diffs, tests, approvals	Low-friction; streaming turns, interruption, short confirmations
Identity and context	Task-local context with explicit workspace selection	Persistent Rasputin identity with bounded recall across approved chats and workspaces
Model policy	Route by mission: coder, researcher, planner, executor	Route by turn: coordinator, speech, memory, action, fallback
Action posture	Preview and approve mutations	Confirm consequential actions; allow safe read-only routines
Output	Artifacts, diffs, test results, receipts	Spoken response, transcript, action summary, receipt

The assistant mode is therefore a policy profile and session experience, not a second permission system. That keeps the workstation safety work valuable as the product becomes more ambient.

3. Current baseline and proposed shift

The existing codebase already provides the main seams needed for this direction: WarSat deploys managed runtimes, the model registry exposes role-based routing, providers normalize model I/O, the governed agent loop executes tasks, MCP tools enforce approvals, and memory remains workspace-aware. The proposal adds typed capabilities, fleet-level lifecycle, audio adapters, a persistent Rasputin context layer, and a local agent-control boundary around those seams.

Current baseline	Proposed evolution
A model is registered with a role and endpoint.	A model is registered as a capability-bearing fleet member with runtime, hardware constraints, health, and fallback policy.
WarSat plans and launches one managed model at a time.	A model pack declares a coherent stack and the fleet manager plans, starts, stops, updates, and diagnoses it as one unit.

Current baseline	Proposed evolution
Catalog can classify speech models but has no speech I/O plane.	STT, VAD, TTS, and wake-word models become first-class local capabilities with streaming health and privacy controls.
Chat and tasks are primarily text-first.	The same session accepts text and audio, emits text and audio, supports interruption, and stores a durable transcript.
Mode routing chooses main/coder/researcher-style roles.	Rasputin preserves the user-facing identity while choosing capabilities, models, agents, tools, and output format from policy and current context.
Context is mostly local to one chat or workspace.	An owner-scoped context index can retrieve approved summaries and references across chats and workspaces with provenance, retention, and deletion controls.
Models expose endpoints but do not own computer control.	A local control broker handles VS Code, shell, tests, Docker, and other host actions; model runtimes receive scoped work requests and structured results only.
Fine-tuning is optional and not required for current workflows.	Personalization starts with prompts, memory, voice, and adapters; fine-tuning is introduced only after data, evaluation, and rollback controls exist.

4. Proposed target architecture

The target is a single local control plane with Rasputin at its center and specialized runtimes behind it. The browser, microphone, schedule, or local client creates an interaction. The session layer turns that input into a normalized turn. Rasputin selects context, models, agents, and tool paths, applies policy, and records an outcome. The model fleet can change without changing the Rasputin identity or conversation contract.

4.1 Rasputin personality and context boundary

Rasputin is the durable application identity across chats, workspaces, and interaction surfaces. A model may speak for Rasputin during a turn, but the model is not the personality's source of truth. Persona, preferences, safety posture, context selection, delegation state, and receipts remain in the local Rasputin control plane.

Cross-chat and cross-workspace recall must be explicit and bounded. Rasputin should retrieve owner-approved summaries, artifacts, and references with workspace provenance, respect visibility and deletion rules, and show why a context item was included. It should never treat every local file or historical conversation as silently available prompt material.

LOCAL COMPUTER BOUNDARY

Rasputin may orchestrate local actions through a broker that exposes narrow capabilities such as opening VS Code, running a named test command, reading an approved workspace, or starting a Docker operation. Model containers do not receive generic host APIs, host filesystem access, credentials, keyboard/mouse control, or unrestricted network access. They return plans and structured results to Rasputin; the local broker enforces scope, approval, and audit.

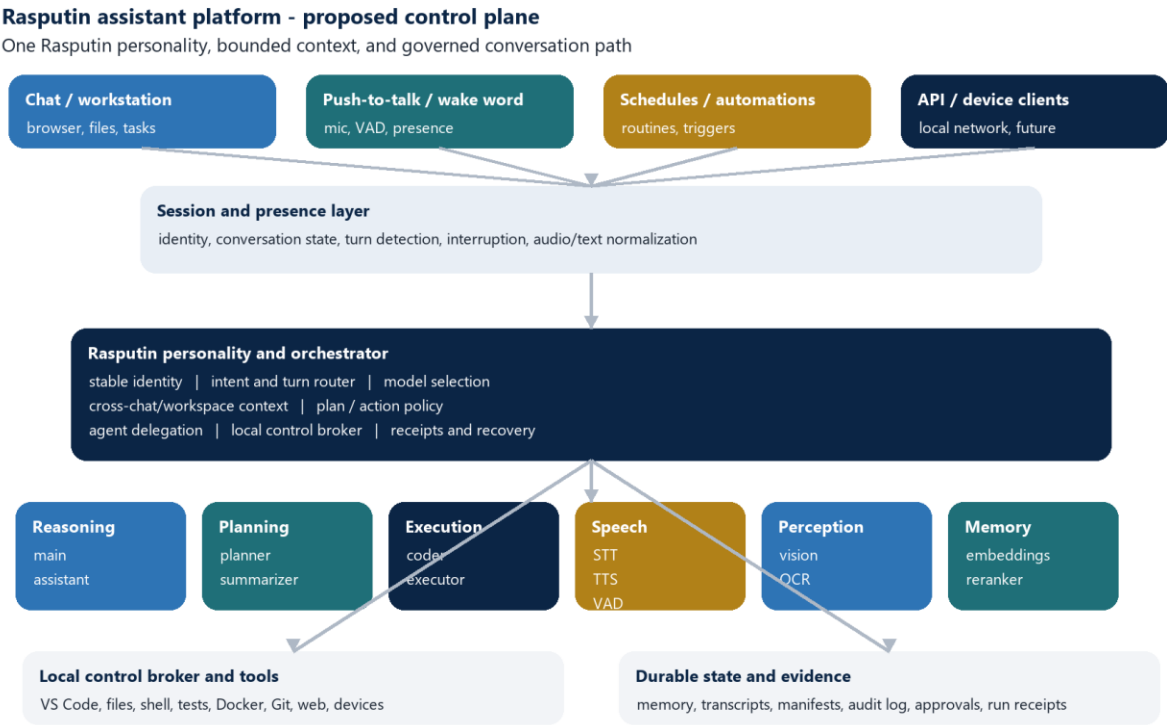


Figure 1. Proposed Rasputin control plane. Rasputin owns identity and policy; worker models receive scoped requests, while the local control broker remains the computer action boundary.

5. Simplifying multi-model launches

The first practical step toward the assistant objective is to make multiple model and agent launches feel like installing one coherent local capability set. The operator should not need to understand ports, container names, GPU selectors, context flags, or runtime-specific arguments for every worker. Rasputin should present one pack, one plan, one status view, and one recovery story.

5.1 Model packs and fleet profiles

A model pack is a versioned, declarative description of one model, agent, or coordinated stack. It is the unit Rasputin installs, reviews, starts, stops, upgrades, backs up, and restores. The pack describes workers; Rasputin remains the stable personality and orchestration surface.

Manifest area	What it declares	Why it matters
Identity	Name, model ID/path, version, license, provenance	Makes the fleet reviewable and portable.
Capabilities	chat, planner, coder, STT, TTS, VAD, vision, embeddings, tools	Routes by proven capability instead of name guessing.
Agent contract	agent role, workspace scope, allowed tools, inputs/outputs, cancellation, approval needs	Lets Rasputin delegate work without giving a worker unrestricted authority.
Runtime	vLLM, llama.cpp, whisper.cpp, faster-whisper, Piper/Kokoro, OpenAI-compatible	Hides runtime-specific launch details behind adapters.

Manifest area	What it declares	Why it matters
Resources	GPU/CPU/RAM/VRAM needs, device affinity, context, concurrency	Allows capacity-aware placement and explains blockers.
Lifecycle	health endpoint, warm-up, dependencies, start/stop order, upgrade policy	Makes a multi-model stack operate as one system.
Routing	preferred roles, latency tier, fallback models, privacy tier	Keeps the assistant responsive when a preferred model is unavailable.
Data policy	cache location, transcript retention, raw-audio policy, network destinations	Prevents a convenience feature from silently changing privacy posture.
Certification	tests passed, tool support, streaming support, speech quality, last verified	Separates observed behavior from catalog assumptions.

5.2 Operator workflow

1. Choose Add capability, Install assistant stack, or Register local agent.
2. Rasputin previews the pack, required downloads, storage, GPU placement, ports, permissions, agent scopes, and expected fallbacks.
3. The fleet planner rejects unsafe or impossible combinations before any container starts.
4. One approval starts the dependency graph in order, then runs health and capability checks.
5. The operator sees one fleet status with per-model details, logs, resource use, and the reason a capability is unavailable.
6. Stop, update, rollback, export, and restore operate on the pack without losing the model registry contract.

PLACEMENT RULE TO PRESERVE

Small models should prefer the largest fitting single GPU. Combined VRAM should be an explicit, runtime-aware decision; on heterogeneous cards, llama.cpp GGUF layer sharding is the safe default for pooled capacity, while vLLM tensor parallelism should require a compatibility check rather than being automatic.

6. Voice input and output foundation

Voice should be treated as a first-class I/O transport for the Rasputin personality and existing conversation/task system, not as a separate assistant brain. Every voice turn should have a normal text representation, a policy decision, a model or agent route, an interruptible response, and a durable receipt.

6.1 Conversation audio path

Stage	Responsibility	Required behavior
Capture	Browser or local client acquires microphone audio.	Visible mic state, permission prompt, device selection, local-only default.

Stage	Responsibility	Required behavior
Presence	Optional wake word, push-to-talk, voice activity detection.	No hidden always-on capture; explicit activation and fast stop.
STT	Speech-to-text runtime produces partial and final transcripts.	Streaming partials, language selection, confidence, timestamps, health.
Turn manager	Detects end-of-turn, interruption, and correction.	Barge-in cancels speech output and preserves the user correction.
Rasputin route	Sends normalized text through the personality and orchestrator.	Same tools, approvals, memory, workspace, agent, and task rules as typed input.
TTS	Text-to-speech runtime streams audio back to the client.	Sentence-level streaming, voice profile, cancellation, fallback to text.
Receipt	Stores transcript, model route, actions, and outcomes.	Raw audio is not retained by default; retention is explicit.

6.2 Voice capability tiers

Tier	Experience	Recommendation
P0 - Push-to-talk	Click or hold a mic button, speak, receive text and optional spoken reply.	First vertical slice. Lowest privacy and reliability risk.
P1 - Hands-free session	Start a voice session, speak naturally, interrupt responses, end explicitly.	Second slice after streaming and barge-in are stable.
P2 - Wake word	Wake Rasputin without touching the browser, with visible listening state.	Only after local wake-word quality, false-positive controls, and privacy UX are proven.
P3 - Ambient routines	Scheduled or context-aware spoken notifications and routines.	Requires presence, notification policy, and stronger safety review.

PRIVACY DEFAULT

The default voice path should retain the final transcript and receipt but discard raw audio after processing. Continuous recording, remote transcription, voice cloning, and device microphone access require separate explicit settings and visible status.

7. The Rasputin runtime: personality, context, and capability routing

The FRIDAY/JARVIS/EVIE objective should be implemented as a governed Rasputin runtime with a stable identity, not as a single model that is expected to contain every skill, permission, and system rule. Rasputin chooses the smallest competent path for each turn, carries approved context across chats and workspaces, delegates to registered agents, and can degrade without pretending a missing capability succeeded.

7.1 Turn contract

1. Receive text, audio transcript, schedule event, or approved connector event.
2. Resolve identity, workspace, presence, mode, latency budget, and privacy tier.

3. Classify intent and select a capability route: answer, plan, research, code, voice, memory, agent delegation, or action.
4. Select a certified model or model chain, with an explicit fallback if the preferred runtime is unavailable.
5. Load the Rasputin persona and build context from bounded conversation and memory services; never expose secrets or unapproved workspace data.
6. Preview any consequential action, delegate to a registered agent or local broker when needed, request approval when required, execute through governed tools, and record the result.
7. Stream a text response and optional speech response; stop immediately on user interruption.
8. Write a receipt containing the user intent, route, tools, approvals, outputs, failures, and follow-up state.

7.2 Agent orchestration and local computer control

Talking to Rasputin should be the operator's control surface for the agent system. Rasputin interprets the request, creates a plan, selects a registered agent and workspace, launches or resumes the work, streams status, handles cancellation, asks for approvals, and returns a concise result with links to artifacts, diffs, and test receipts.

The computer bridge is a local process boundary, not an API model capability. The broker can expose narrow operations such as open VS Code, run a named test command, inspect an approved workspace, or start a Docker action. It should use allowlists, scoped credentials, dry-run previews, emergency stop, and complete receipts. Models only receive the minimum task input and structured output required for their role.

AUTHORITY MODEL

Rasputin is the authority and user-facing personality. Agents and model containers are workers. The local broker is the only component allowed to turn an approved plan into host-computer effects, and every mutation remains subject to the existing Rasputin approval and audit rules.

7.3 Proposed capability roles

Capability	Typical responsibility	Can act?	Fallback
Rasputin personality	Maintain persistent identity, select approved context across chats/workspaces, and choose routes.	No direct mutation	Safe text response
Agent supervisor	Launch, monitor, cancel, and summarize registered agents within approved workspaces.	Yes, gated	Plan-only mode
Planner	Break work into steps, identify dependencies and approvals.	No direct mutation	Rasputin personality
Coder / executor	Edit, test, repair, and report in an isolated workspace.	Yes, governed	Plan-only mode

Capability	Typical responsibility	Can act?	Fallback
Researcher	Search approved sources and produce cited evidence.	Read-only	Local knowledge / ask user
STT / VAD	Turn audio into text and detect speech boundaries.	No	Text input
TTS	Turn approved response text into audio.	No	Text response
Memory	Summarize, retrieve, and retain bounded personal/workspace context.	No	No-memory turn
Device / connector	Perform external actions through explicit scopes.	Yes, gated	Draft or preview only

7.4 Fine-tuning and identity

A fine-tuned personal model is acceptable, but it should be a later optimization rather than the foundation of the first assistant slice. Begin with a declarative Rasputin persona profile, bounded cross-workspace memory, tool policies, voice profile, agent contracts, and a model certification suite. These are easier to inspect, change, roll back, and keep private than a large opaque fine-tune.

- Do not encode permissions, safety rules, or approval requirements solely in model weights.
- Collect opt-in traces only after the operator can inspect, redact, export, and delete them.
- Fine-tune behavior and style, not hidden credentials or irreversible action habits.
- Use LoRA or adapters first so the base model, persona, and mission styles remain separable.
- Gate promotion on coding, tool-use, voice, refusal, privacy, and regression evaluations.

8. Safety and trust boundaries

The assistant objective increases the importance of the current safety work. A voice interface and natural-language agent request can make actions feel casual even when they are consequential. The design must therefore preserve the existing governed action plane, keep model runtimes away from generic host access, and add presence-aware controls without weakening approvals.

Risk	Required control	Approval gate
False wake or mistaken transcription	Show transcript before high-impact action; allow correction and cancel.	Always for external or destructive actions.
Prompt injection through speech, files, or connectors	Treat all content as untrusted; keep tool policy outside model text.	Policy check before tool execution.
Accidental device or network action	Device scopes, dry-run preview, allowlist, emergency stop, audit receipt.	Explicit approval and current presence.
Privacy leakage through audio or memory	Local default, raw-audio expiry, owner/workspace scope, export/delete controls.	Separate consent for retention or remote processing.
Uncontrolled model-to-computer access	Local broker only, narrow allowlists, scoped credentials, dry-run, emergency stop, and structured receipts.	No generic model API to the host; explicit approval for mutations.
Model fleet failure or drift	Health checks, capability certification, fallbacks, rollback, per-model status.	No route to unhealthy or uncertified capability.
Over-automation and loss of control	Assistant routines are bounded, scheduled, budgeted, and interruptible.	User enables each routine class.

NON-NEGOTIABLE

The assistant may sound autonomous, but it must never be trusted merely because it speaks confidently. Authority comes from the same explicit user, workspace, connector, approval, and audit controls that govern workstation actions.

9. Phased implementation proposal

The phases below are deliberately incremental. Each phase produces a useful capability without requiring the full assistant vision to be complete. The first two phases should be implementable without committing to a specific persona model or voice vendor.

Phase	Outcome	Primary deliverables	Exit evidence
0. Contract and decisions	Approve the direction and freeze the first interfaces.	Rasputin identity and context contract, model-pack shape, agent contract, local broker boundary, assistant session contract, privacy defaults, decision log.	Maintainer approval of this proposal and an implementation slice.
1. Fleet foundations	Launching several models and agents becomes one reviewed operation.	Model-pack manifest, fleet planner, dependency graph, capacity checks, grouped lifecycle, capability health, agent scopes, fallback routes.	Install/start/stop/update a small assistant pack with explainable status and no manual port editing.
2. Audio foundation	Rasputin can receive and produce local audio safely.	Audio device permissions, push-to-talk capture, VAD/STT/TTS adapters, streaming health, transcript storage and expiry policy.	A local audio round trip passes with raw-audio retention off by default.
3. Rasputin context and talk/listen	One personality can carry bounded context across approved chats/workspaces and hold a live voice turn.	Context index, provenance, retrieval, partial transcript, streamed TTS, barge-in, correction, text fallback, voice profile.	Rasputin answers a new chat using an approved prior context reference and supports interruption.
4. Agent and computer control	Rasputin can delegate useful work without handing models the computer.	Agent supervisor, local control broker, VS Code adapter, named test runner, schedules, routine previews, budgets, receipts, emergency stop.	Rasputin starts a registered agent, opens VS Code or runs a named test through the broker, and pauses mutations for approval.
5. Personalization	Rasputin becomes recognizably yours without losing control.	Persona profile, preference and context memory, opt-in trace dataset, adapter/fine-tune pipeline, regression suite, rollback.	Personalized behavior improves against a held-out evaluation set without safety regression.

10. Decisions requested from the maintainer

The proposal is intentionally incomplete until these choices are approved. The recommended defaults are designed to keep the first implementation small, local, and reversible.

Decision	Recommended starting point	Approve / revise
Rasputin identity	Rasputin is the stable user-facing personality and context authority; models remain replaceable workers.	
Cross-workspace context	Opt-in, owner-scoped retrieval with provenance, visibility controls, export/delete, and no implicit secret sharing.	
Agent orchestration	Talk to Rasputin to plan, delegate, monitor, cancel, and summarize registered agents; approvals gate mutations.	

Decision	Recommended starting point	Approve / revise
Computer control boundary	A local broker may open VS Code and run named tests; model runtimes receive no generic host API, filesystem, shell, or credential access.	
Voice activation	P0 push-to-talk first; wake word is opt-in later.	
Audio retention	Final transcript and receipt retained; raw audio discarded by default.	
Cloud fallback	Local-only by default; remote fallback visibly disabled unless enabled in Safety.	
Speech runtime	Adapter boundary first; select one local STT and one local TTS implementation after hardware tests.	
Assistant model	Use an existing certified local reasoning model first; defer fine-tuning until evaluation data exists.	
Model-pack scope	Start with coordinator + coder + STT + TTS + embeddings; add vision and devices later.	
Device control	No device mutations in the first voice slice; read-only status and explicit previews only.	
Multi-user	Keep owner/workspace scoping; assistant persona and voice preferences remain per owner.	
Target interaction	Browser push-to-talk and typed chat share one session contract.	

11. Proposed first vertical slice after approval

The first implementation should be narrow enough to verify end to end and broad enough to prove the architecture. It should not attempt wake-word automation, broad device control, or fine-tuning in the same change.

1. Add typed capability metadata and a model-pack manifest without removing the current role-based registry fields.
2. Implement a fleet plan preview that explains placement, dependencies, ports, health checks, and fallbacks before launch.
3. Add the Rasputin personality and context contract, with owner/workspace-scoped retrieval from one approved prior chat and visible provenance.
4. Add a push-to-talk session contract with an adapter interface for local STT and TTS; use text fallback when either runtime is unavailable.
5. Route a spoken turn through Rasputin's governed chat/task path and record a transcript plus run receipt.
6. Register one agent and local control broker path that can open VS Code and run one named test in an approved workspace; return structured results only.
7. Demonstrate one safe read-only assistant routine and one approval-gated workstation action.
8. Verify with isolated tests, a live local model, a live speech round trip, a cross-chat context check, and an explicit privacy/control audit.

DEFINITION OF DONE FOR THE SLICE

The operator can talk to Rasputin, receive a response that uses an approved context reference from another chat or workspace, hear and interrupt the answer, and ask Rasputin to delegate a task. Rasputin can open VS Code or run a named test through the local broker, while the system clearly reports which model or agent handled each step and never gives a model runtime generic access to the computer.

12. What this proposal deliberately does not decide

- The final voice, visual presentation, mannerisms, and degree of fictional inspiration; the Rasputin identity itself is the chosen product direction.
- A specific STT, TTS, wake-word, or foundation model before hardware and quality trials.
- An always-listening microphone policy or raw-audio archival policy.
- Which additional household, operating-system, financial, or external communication capabilities should be exposed after the local broker proves safe.
- Which chats and workspaces are approved for cross-context recall by default.
- The initial registered agent catalog and editor/test adapters beyond the first vertical slice.
- A public multi-tenant hosted offering; the near-term product remains self-hosted and local-first.
- A fine-tune dataset, training recipe, or model license choice before provenance and evaluation are settled.

Appendix A. Current implementation anchors

This proposal is grounded in the current Rasputin boundaries rather than a parallel greenfield application:

Current boundary	Relevant source of truth	Role in the proposal
Model registry and routing	backend/models/registry.py	Preserve role routing while adding typed capabilities, health, and pack identity.
Provider I/O	backend/models/providers.py	Add speech/session adapters beside existing chat and streaming provider normalization.
Managed model deployment	backend/warsat/ and backend/api/warsat_api.py	Evolve WarSat into a fleet planner and lifecycle manager.
Agent and task loop	backend/engine/agent.py and backend/api/agent.py	Keep governed planning, tool calls, budgets, workspace scope, and receipts.
Tool and approval layer	backend/mcp/layer.py and backend/core/security.py	Remain the authority for filesystem, shell, Docker, network, local computer broker, and future device actions.
Memory and context	backend/rag/ and backend/engine/context.py	Extend bounded, owner-scoped context to Rasputin identity, cross-chat/workspace recall, preferences, and voice-session memory.
Local agent and computer bridge	backend/engine/agent.py, backend/mcp/layer.py, and local control adapters	Let Rasputin delegate to registered agents and host actions without exposing generic computer access to model runtimes.
Current speech signal	backend/models/catalog.py	Catalog recognizes speech model purpose, but speech transport and runtime integration remain future work.

Appendix B. Review checklist

Before implementation begins, mark the proposal as approved, approved with changes, or deferred. The next iteration should update this document rather than silently changing the scope in code.

- The north-star product direction is correct: Rasputin as a persistent personality and context authority over the workstation.
- Model packs and fleet planning are the right first simplification for multi-model and agent launches.
- Cross-chat/workspace recall is bounded, owner-scoped, provenance-aware, and explicit enough to trust.
- Talking to Rasputin is the right control surface for planning and orchestrating agents.
- The local computer broker boundary is clear: model runtimes do not receive generic host API access.
- Push-to-talk is the right first voice tier; wake word and ambient behavior wait for later review.
- Raw-audio retention, cloud fallback, device control, and fine-tuning defaults are acceptable.
- The proposed capability roles and assistant turn contract are understandable and complete enough to implement.
- The first vertical slice is appropriately bounded for a safe, reviewable iteration.

APPROVAL STATE

Pending maintainer review. No implementation is authorized by this document alone.